COMPSCI 130 S1 2022
Introduction to Software Fundamentals

Binary Trees – Processing

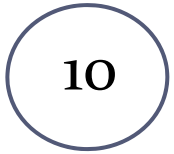
Agenda & Reading

- Agenda
  - Nest list representation of a binary tree
  - Recursive functions on binary trees:
    - Sum of nodes
    - Number of leaves
    - Height of tree

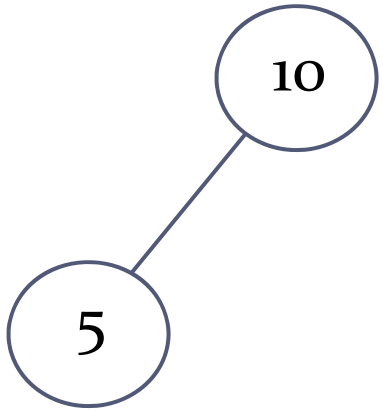
- Online Coursebook:
  - https://onlinetextbook.auckland.ac.nz/19-binary-trees/binary-tree-processing/

Printing a binary tree

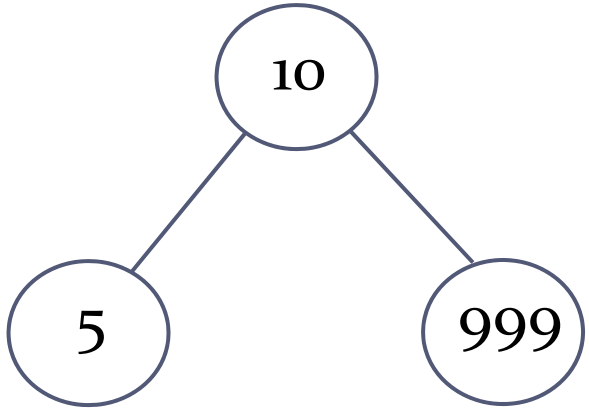
- Tree traversals give us a systematic way of accessing each node in the tree
- But we lose information about the structure of the tree when we use a traversal to print the nodes
  - How can we print the tree in a way that maintains structural information about the tree?
    - One approach is to print the tree as a list consisting of three elements (i.e. this is the "nested list" approach):
      - root, left tree, right tree
    - where the left and right trees are either None, or another list representing the respective tree, thus following the recursive definition




```
['10', None, None]
```



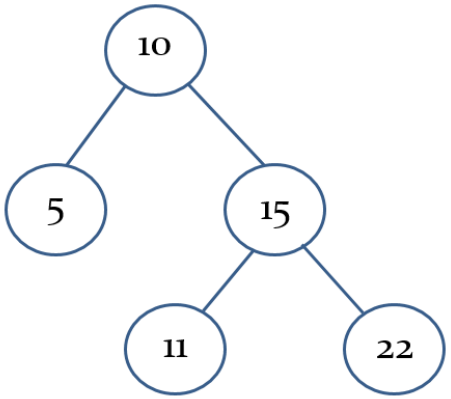

```
['10', ['5', None, None], None]
```




```
['10', ['5', None, None], ['999', None, None]]
```

A "nested list" representation

- Maintains the *structure* of the tree in an ordinary list



Nested list:

```
[10, [5, None, None], [15, [11, None, None], [22, None, None]]]
```

Generating a nested list representation

- The following function will convert a binary tree to a nested list

```
def convert_tree_to_list(tree):
    if tree == None:
        return None
    else:
        result = []
        result.append(str(tree.get_data()))
        result.append(convert_tree_to_list(tree.get_left()))
        result.append(convert_tree_to_list(tree.get_right()))
        return result
```

Printing vertically (with spaces)

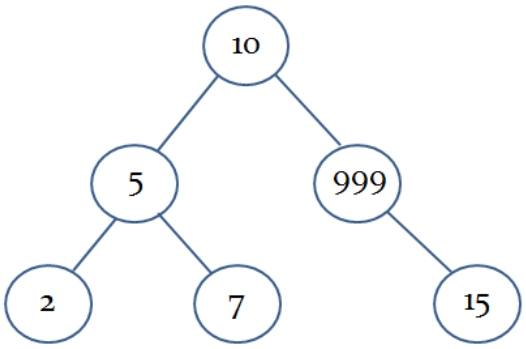
- Or we could try to print the tree vertically:

```
def print_tree(tree, level):
    print('  ' * (level*4) + str(tree.get_data()))
    if tree.get_left() != None:
        print('(L)', end = '')
        print_tree(tree.get_left(), level + 1)
    if tree.get_right() != None:
        print('(R)', end = '')
        print_tree(tree.get_right(), level + 1)
```

Print the root node

Print the left subtree

Print the right subtree

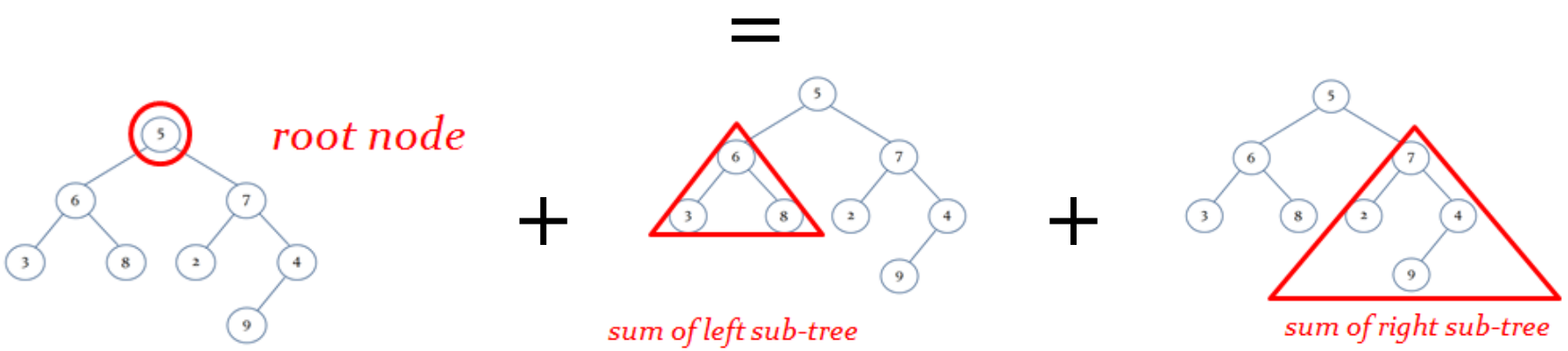



```
print_tree(t, 0)
```

```
10
(L)    5
(L)        2
(R)        7
(R)    999
(R)        15
```

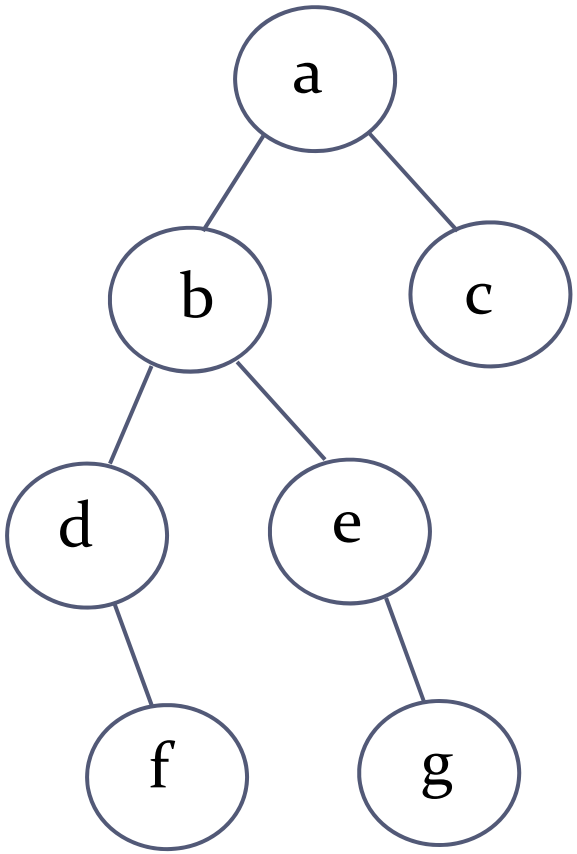
Recursive Tree Functions – Sum of Values

- Process the left and right subtrees using recursion
  - Example: calculate the sum of the values in the tree




```
def sum_tree(tree):
    if tree == None:
        return 0
    else:
        total = tree.get_data()
        total += sum_tree(tree.get_left())
        total += sum_tree(tree.get_right())
        return total
```

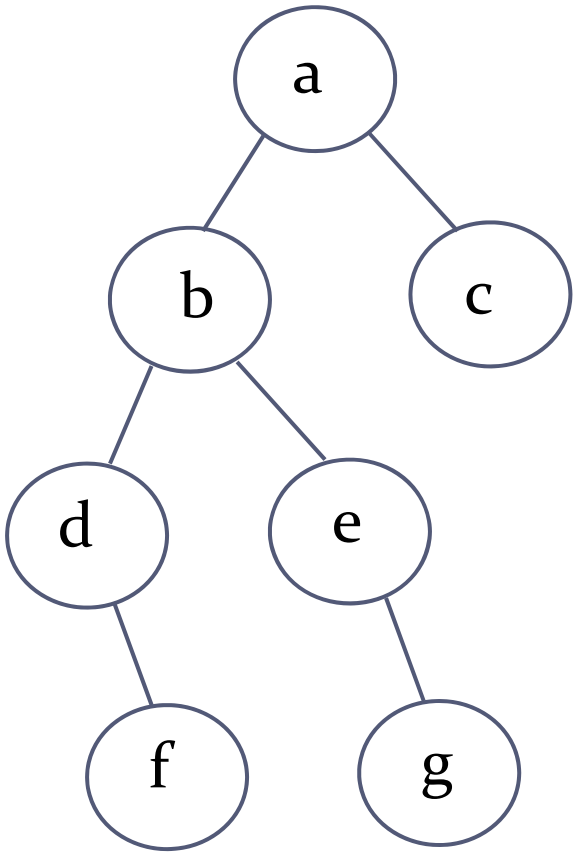
Recursive Tree Functions – Number of Leaves



Number leaves: 3

```
def num_leaves(tree):
    if tree == None:
        return 0
    else:
        return max(1, num_leaves(tree.get_left()) +
                      num_leaves(tree.get_right()))
```

Recursive Tree Functions – Tree Height



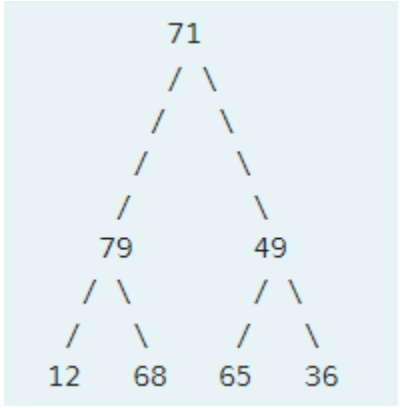
Height is defined as the maximal distance to the root node. So for a tree consisting solely of a root node, the height is zero!

Here the height is: 3

```
def height(tree):
    if tree == None:
        return -1
    else:
        return 1 + max(height(tree.get_left()),
                       height(tree.get_right()))
```

Exercise 6

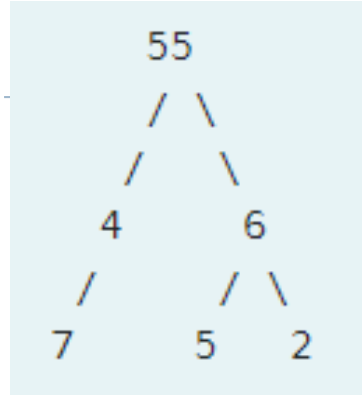
- Consider the following binary tree:



- What is the nested list representation of the above binary tree?

Exercise 7

- Consider the following binary tree:



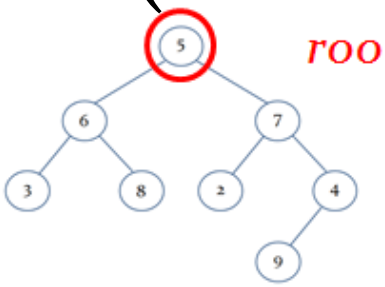
- Return all paths to the leaves of the above binary tree. Your answer is a list of lists of node values, where each sublist starts at the root.

Exercise 8

- Define a function named get_max (b_tree)
  - finds the maximum integer of all elements in the binary tree recursively.
  - returns -999 if the tree is empty

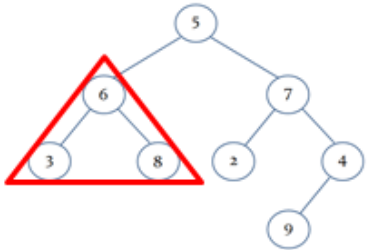
```
def get_max(b_tree):
 if b_tree == None:
        return -999
    else:
```

max(



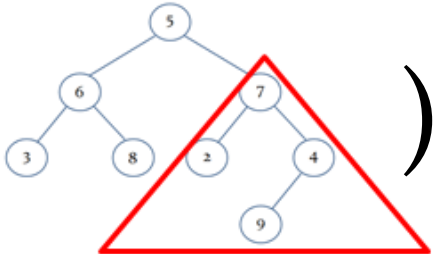
root node

,



max of left sub-tree

,



max of right sub-tree

)